Loom AI: A Multi Agents Calendar Planning Agent

Hung-Kai Huang (hh3164) · Yipeng Wang (yw4623)

STAT GR5293: Generative AI and LLMs · Final Project Report · May 2026

1. Introduction

Two situations come up constantly. Someone has a goal that fits in a single sentence, like "learn enough SQL to pass a junior data analyst interview," but their calendar already looks like Tetris. The work of converting that sentence into a sequence of dated events is fundamentally tedious: estimate durations, sequence prerequisites, find free slots, dodge the Monday-morning meeting block. And no existing AI tool actually does this. They expect you to type the events in yourself and then they shuffle the result.

Loom AI closes that gap. You fill out a short intake form describing your goal, deadline, working hours, and energy levels by time of day. The agent decomposes the goal into subtasks, packs them into free slots around your existing events, and returns three distinct candidate schedules. Each candidate is validated against hard constraints, annotated with a short rationale, and waits behind an explicit approval step before anything hits your calendar.

The interesting design question was not really about LLMs. It was about where to stop using them. LLMs are genuinely good at turning a vague goal into a sensible list of subtasks, critiquing that list, and writing short explanations. They are not good at time geometry: they hallucinate overlaps, drift on durations, and fail at the kind of bin-packing arithmetic that calendar scheduling actually needs. So we gave each component what it is actually suited for. The LLMs handle language; deterministic Python handles math. A LangGraph state machine keeps the two halves talking to each other.

That split leads to two research questions that shaped every design decision in this project:

- What does division look like between LLM and deterministic code for our project?
- What contract should the LLM emit so that downstream Python schedulers can respect dependencies, complexity, and ordering without re-querying the model?

2. Background and Related Work

2.1 Who Actually Needs This

Three kinds of users kept coming up when we thought about who Loom AI is for. Students are the most obvious: cramming for a midterm, prepping for a technical interview, scoping a final project in the days before a deadline. Professionals come next, specifically anyone trying to fit a learning or deliverable goal around a calendar that is already half meetings. And then there are casual planners, someone organising a trip or a life event with a natural deadline. What unites all three is that they start with a sentence, not a list, and they need help turning it into dated events.

2.2 What Existing Tools Actually Do

We surveyed three AI calendar products to check whether this problem was already solved.

Product	What it actually does	Goal decomposition	Constraint validation	Strategy choice
Reclaim.ai	Moves tasks within user-defined windows	No	Yes	No
Motion	Shuffles tasks around conflicts	No	Yes	No
Fantastical	Parses natural-language event text	No	No	No

All three share the same assumption: the user has already done the cognitive work of breaking the goal into events, and the tool optimises over whatever list it receives. None of them decompose a goal, validate against hard constraints, or offer you a choice of strategies. Those three gaps are exactly what Loom AI is built to fill.

2.3 Why LangGraph

Calendar planning has a natural structure that makes plain Python orchestration awkward. There is a fan-out (multiple strategies designed in parallel), a fan-in (a validator consuming all three results), and a mid-run pause (the human approval step). LangGraph handles all three cleanly.

On the structural side, LangGraph forces a typed state machine. One AgentState dict carries every field, and each node documents what it reads and writes, so the data contract is explicit and auditable. On the execution side, parallel branches with reducers let the three scheduling heuristics run concurrently and write to a shared debug trace without race conditions. And the checkpointed pause means the graph can stop cold at the human approval node and resume exactly when the user clicks a button, without any polling or state serialisation overhead on our side. We did not use tool-calling features in this project, but those three orchestration affordances alone justified the dependency.

3. System Architecture

3.1 The Pipeline at a Glance

The full system is an eight-agent LangGraph with one optional revision branch and one parallel fan-out. Figure 1 shows the complete topology, colour-coded by node type.

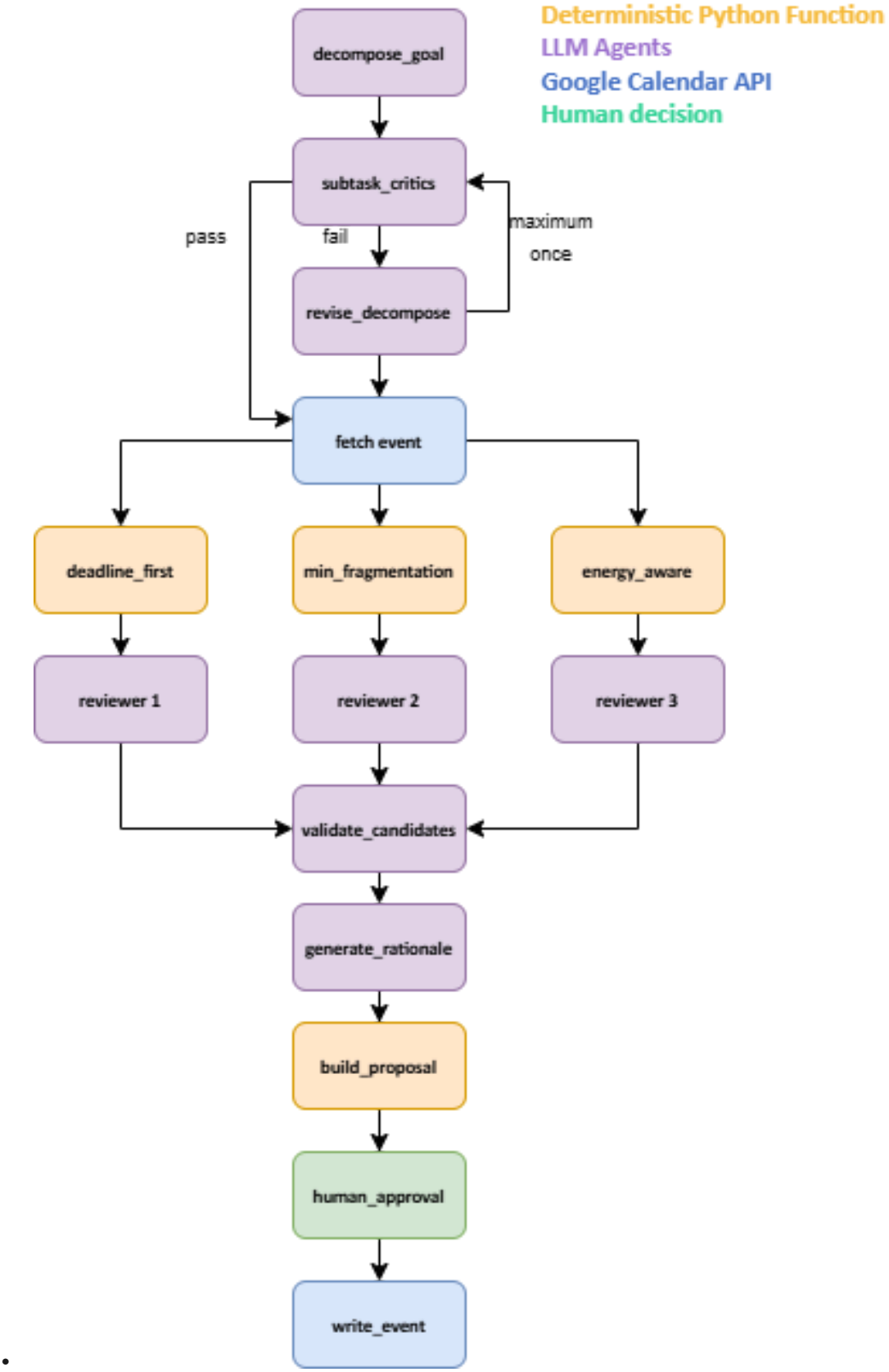


Figure 1. End-to-end pipeline

3.2 How the Nodes Fit Together

The non-agent nodes are governed by four design rules:

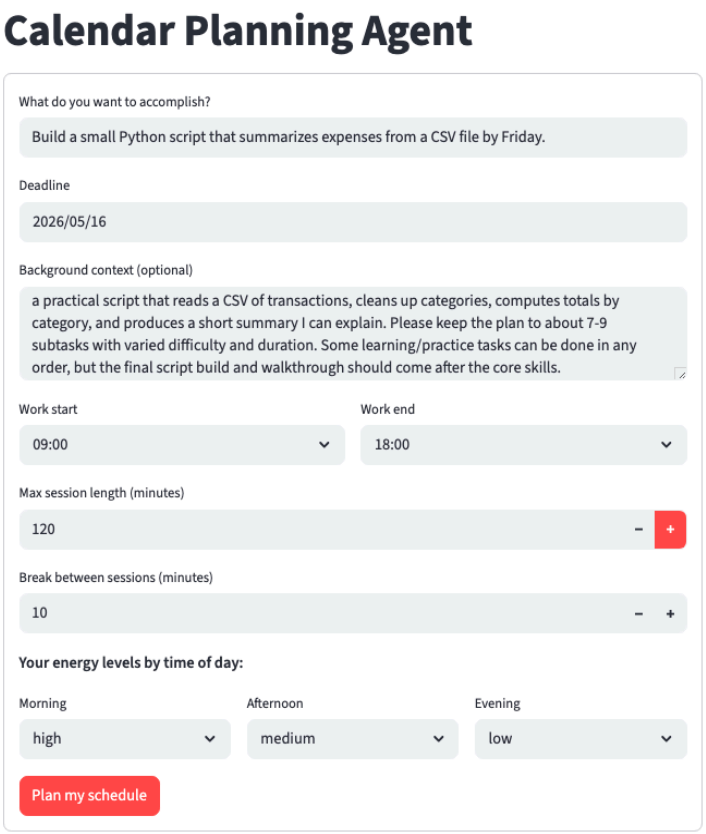
Principle	Behavior	Effects/Constraints
State-driven	A single AgentState TypedDict is threaded through every node; each node documents which fields it reads and which it writes.	Data flow is explicit and inspectable; no hidden side channels between nodes.
Pure functions for math	Heuristics and validators contain no LLM calls and no persistence.	Deterministic and unit-testable in isolation.
Validator Authority	Reviewers may score and recommend, but only the validator's verdict counts.	Single authoritative pass/fail signal for acceptance.
Calendar write	The calendar node calls events().insert() exclusively.	No code path updates or deletes existing user events.

The agents are operating harmoniously:

Principle	Behavior	Effects/Constraints
Decomposer	Turns the goal into a list of subtasks	Sequential, single pass
Critic	Checks the decomposition for logical soundness and practical schedulability	Single critique round
Reviser	Revises the decomposition if the critic flagged a problem	Runs at most once
Reviewer A	Produces a candidate schedule; scored on 4 dimensions (deadline, fragmentation, energy-aware, feasibility)	Run in parallel
Reviewer B		
Reviewer C		
Validator	Checks candidates against 4 hard constraints	Pure Python, zero LLM
Rationale generator	Produces one short rationale per candidate	Output surfaced for human approval

3.3 The Intake Form

Everything starts with a single Streamlit form.



The form has seven fields.

Goal and **deadline** are self-explanatory.

Background context is optional free text for anything the decomposer should know about the situation.

Work start and **work end** bound the validator's out-of-hours check.

Max session length caps any single event's duration.

Break length inserts a gap between sessions.

Energy levels (morning, afternoon, evening; high, medium, or low) feed directly into the energy-aware scheduling heuristic.

3.4 Goal Decomposition: Three Structural Tags

The decomposition prompt is the most load-bearing LLM surface in the pipeline. Every downstream scheduler depends on the tags it elicits.

- `[group:X]` partitions subtasks into logical learning phases. Phases execute in first-appearance order, enforcing dependency without the LLM having to think about calendar slots.
- `[shuffle:yes|no]` declares that tasks within a group are peer-equivalent and can be freely reordered. This is how min_fragmentation and energy_aware get room to do interesting things.
- `[complexity:low|med|high]` must be consistent with the task's duration. The energy-aware heuristic reads this to decide which time-of-day slot is a good fit.

These three tags are the architectural keystone of the project. They are compact enough to fit in a single prompt and expressive enough to drive three different schedulers with no further LLM involvement.

3.5 Critic and Reviser

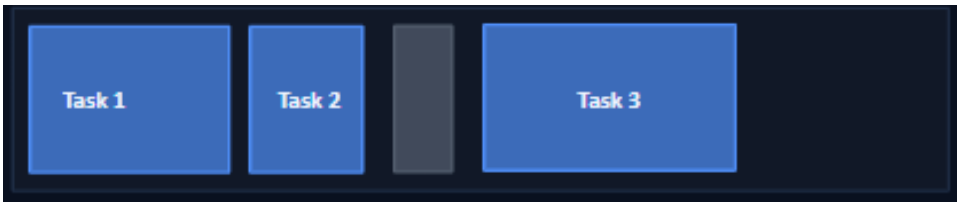
After decomposition, the subtask_critic node reads the list and looks for real problems: tasks that are too vague to put on a calendar, missing preparation or review steps, durations that do not match the stated complexity, or ordering within a group that would break learning dependencies. It returns a structured result with a passed flag, an issues list, and a revision_instruction.

If passed is false, the graph routes through revise_decompose once, re-prompting the original decomposer with the critic's instruction appended. The loop is hard-bounded at one revision. The critic's prompt also explicitly says: "Do not be picky about style. If the plan is schedulable and useful, pass it." That instruction prevents the critic from becoming a bottleneck that blocks reasonable plans.

3.6 Three Scheduling Strategies

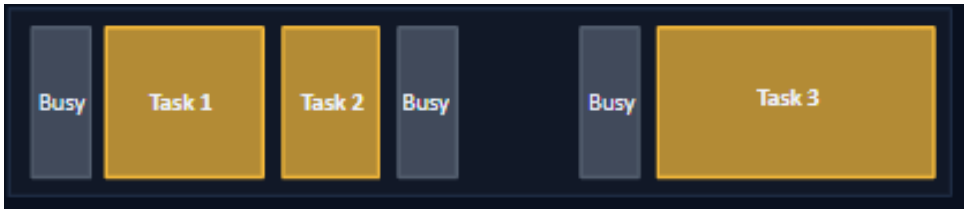
All three heuristics share the same function signature: (subtasks, free_slots) returns ProposedEvents.

3.6.1 deadline_first



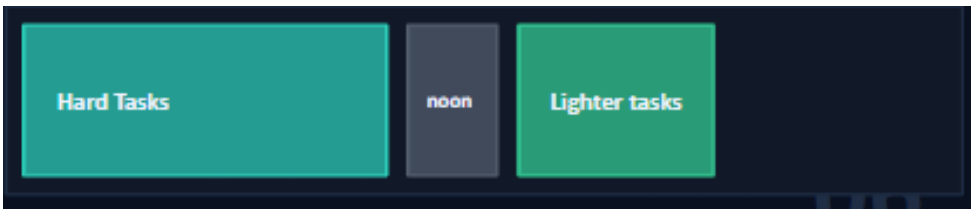
Each subtask drops into the first available slot that fits. Longer tasks within a shuffle-yes run claim earlier viable slots. A monotonically growing min_allowed_start variable prevents calendar order from silently inverting dependency order.

3.6.2 min_fragmentation



Matches the longest remaining task to the largest available slot on the earliest eligible date. The idea is that heavy cognitive work should live in long uninterrupted blocks, and lighter tasks should fill the gaps. Compacted, not scattered.

3.6.3 energy_aware



Each time-of-day window (morning, afternoon, evening) gets a score from the user's energy profile, and the scheduler finds the slot that minimises the mismatch between a task's complexity and the energy available when it would be scheduled.

3.7 Validator: Four Hard Constraints

The validator runs after every heuristic. It is about thirty lines of pure Python, has zero LLM dependence, and checks four things:

1. OVERLAP: candidate event overlaps an existing busy block
2. SELF_OVERLAP: two candidate events overlap each other
3. OUT_OF_HOURS: event falls outside the working window
4. DEADLINE_EXCEEDED: event ends after the deadline

Every overlap check uses the standard interval test: start_a < end_b AND start_b < end_a.

One nuance worth making explicit: a candidate that fails validation is not discarded. Violations only occur when there is genuinely no way to fit a task given the existing constraints, which is useful information the user should see. The frontend surfaces violations next to each candidate. The reviewer agents may score and recommend, but they are explicitly subordinate to the validator: no reviewer can recommend a strategy with hard violations unless every strategy has them.

3.8 Review, Rationale, and Human Approval

Three separate LLM reviewers scores validated candidates on four dimensions.

- Deadline: how well the plan protects the deadline.
- Energy: how well task complexity is matched to the user's energy profile.
- Fragmentation: how well the strategy preserves large usable focus blocks.
- Feasibility: how realistic is a plan for a real person to actually follow.

Each returns a 1-to-10 score per strategy, a one-sentence comment, and an overall recommendation.

generate_rationale then produces one short paragraph per candidate. If the LLM fails or hits a token limit, the system does not retry; it falls back immediately to a deterministic generator that produces a short summary from event count, violation count, and the strategy's general posture. This fallback is not a nice-to-have: without it, a single rationale hiccup blocks the user from seeing any results.

build_proposal_node detects near-duplicates by hashing each candidate's sorted event tuples. When all three hashes match, the UI collapses into a single approve/reject view rather than showing three identical plans. The graph then pauses at human_approval. On approval, the chosen candidate is written to Google Calendar via events().insert() only. On rejection, the graph ends and nothing is written.

3.9 Why We Did Not Fine-Tune

Most course projects centre on fine-tuning a model for a specific task. We did not, and it is worth explaining why. The range of user goals in calendar planning is enormous: a CS interview, a wedding, a novel, a marathon programme. Fine-tuning on any one slice hurts the others, and labelling a representative cross-section is expensive. A strong general pretrained model gives us breadth: if it produces a correctly tagged subtask list, all the downstream scheduling logic is goal-agnostic. The

multi-agent setup is our substitute for fine-tuning. Stacking critique layers on a general model often beats retraining on narrow data when the input distribution is too wide to label cheaply.

4. Implementation Notes

4.1 Tech Stack

Layer	Technology
Frontend / UX	Streamlit (intake form, three-column candidate display, calendar widget, debug panel)
Agent workflow	LangGraph 1.0+ (typed state, parallel branches, conditional edges, checkpointed pauses)
Planning LLM	gemini-3.1-pro-preview (decomposition, critic, reviser)
Review + Rationale LLM	Grok llama-3.3-70b-versatile (four reviewers, rationale generator)
Scheduling	Pure Python deterministic heuristics (no LLM calls)
Calendar backend	google-api-python-client + google-auth-oauthlib (live); in-process mock (offline)
Testing	pytest 8 (markers gate the slower real-LLM suite)

We use two different model providers deliberately. If one misbehaves in production, the other half of the pipeline keeps functioning. Swapping a model is a one-line change in the .env file.

4.2 LLM Output Repair

The most common decomposer failure during development was wrapping JSON in markdown fences, making the output unparseable. The _clean_llm_json function handles this, along with smart-quote normalisation, trailing comma removal, and Python literal conversion, before the JSON parser ever sees the payload.

```
def _clean_llm_json(text: str) -> str:
    text = re.sub(r"^```(?:json)?\s*|\s*```$", "", text.strip(), flags=re.M)
    text = text.translate(str.maketrans({
        '\u201c': '"', '\u201d': '"', '\u2018': "'", '\u2019': "'"
    }))
    text = re.sub(r",(\s*[}\]])", r"\1", text)   # trailing commas
    text = re.sub(r"\bTrue\b", "true", text)       # Python -> JSON literals
    text = re.sub(r"\bFalse\b", "false", text)
    text = re.sub(r"\bNone\b", "null", text)
    return text
```

If json.loads still fails after cleaning, a last-resort scan finds the first decodable JSON value starting from any [or { character. Over 99 real LLM integration calls, the parse rate is 100%.

4.3 Validator Core

The validator is small enough to show in full:

```
def _overlaps(a_start, a_end, b_start, b_end) -> bool:
    return a_start < b_end and b_start < a_end

def validate_schedule(events, busy_blocks, work_start, work_end, deadline):
```

```
    violations = []
    for e in events:
        for b in busy_blocks:
            if _overlaps(e.start, e.end, b.start, b.end):
                violations.append(Violation(e.name, 'OVERLAP', ...))
        if e.start.time() < work_start or e.end.time() > work_end:
            violations.append(Violation(e.name, 'OUT_OF_HOURS', ...))
        if e.end > deadline:
            violations.append(Violation(e.name, 'DEADLINE_EXCEEDED', ...))
    for a, b in itertools.combinations(events, 2):
        if _overlaps(a.start, a.end, b.start, b.end):
            violations.append(Violation(a.name, 'SELF_OVERLAP', ...))
    return ValidationResult(passed=len(violations) == 0, violations=violations)
```

5. Evaluation and Results

5.1 Test Architecture

The repository contains 317 tests in three tiers. All pass.

Suite	Tests	Runtime	LLM called?	What it covers
Programmatic unit	118	~1s	No (mocked)	Validator, free-slot computation, mock calendar, JSON repair
Pipeline unit	100	~1s	No (mocked)	Heuristic correctness, tag handling, rationale fallback
LLM integration	99	~113s	Yes	Structural assertions on every LLM-facing node
Total	**317**	**~115s**		**All passing**

The fast suite (programmatic + pipeline, 218 tests) needs no credentials and runs in about two seconds on every commit. The integration suite gates on Vertex AI credentials and runs nightly.

5.2 Real-LLM Testing on Mock Calendars

The 99 integration tests are the evaluation surface that actually matters, since they are the only ones that exercise real model behaviour. We created three synthetic calendar topologies as fixtures: CALENDAR_SPARSE (mostly empty workdays), CALENDAR_MORNING_HEAVY (morning slots busy), and CALENDAR_AFTERNOON_HEAVY (afternoon slots busy). Tests split across five categories.

Test category	Count	Calendars covered	What it checks
Decomposition structural	30	All three	Tags present, subtask count within bounds, durations within max session
Critic behaviour	17	N/A (logic test)	Passes good plans, flags flawed ones, no false critical issues on the worked example

Test category	Count	Calendars covered	What it checks
Rationale generation	25	All three	Three distinct rationales, word cap respected, no fallback error markers present
Revision correctness	15	N/A (logic test)	Tasks flagged by the critic no longer appear in revised JSON output
Heuristic alignment	12	All three	Each heuristic produces at least one valid candidate per calendar topology
Total	**99**		**All passing**

The economic story here is what we are most proud of. Every test file uses pytest fixtures with scope="session": each scenario calls the real LLM once, and that response is reused across every structural assertion in the file. Nine real API calls power 99 tests, an amplification ratio of about 11x. That is what makes a real-LLM suite cheap enough to actually run.

Because LLM responses are non-deterministic, no test asserts an exact string. Every assertion is structural: correct shape, count within bounds, correct types, hard limits respected, required tags present, correct severity calibration, distinct rationales, length within cap, no error-signal strings. For revision tests, we verify that the task name the critic flagged is absent from the revised JSON.

5.3 User Study (n = 20)

We ran a user study with 20 graduate student volunteers. Each submitted a real personal goal, reviewed the three candidate schedules, and answered three questions: was the plan better than what they would have done manually, how much time did it save, and what felt wrong.

17 of 20 (85%) gave the proposal 4 or 5 stars. Goals spanned self-learning (interview prep, picking up a new framework), project deliverables (a methodology write-up, a small script), and leisure pursues (writing a novel, making a documentary).

Three feedback themes emerged consistently. First: the time savings are real, and the proposal quality is good enough to accept. Second: where the LLM has solid domain knowledge (SQL, Python, common interview topics), the subject decomposition is accurate. Third: on genuinely open-ended creative goals, like "write a novel," the LLM under-estimates the time needed for brainstorming and reflection. That third point suggests energy level is not the only signal we should be collecting. A confidence or capability field, capturing how comfortable the user already is with the topic, would help the energy-aware heuristic make better duration choices on unfamiliar work.

5.4 Ablation: Do the Three Strategies Actually Differ?

We ran the same goal (seven subtasks derived from "build a Python script that summarises expenses from a CSV by Friday") through all three heuristics on all three synthetic calendars, using one fixed energy profile (morning high, afternoon medium, evening low). Across 9 cells, the heuristics produced visibly different schedules in 6. The remaining 3 were correctly flagged as identical by build_proposal_node, collapsing the UI accordingly. When the calendar has room for different solutions, the user gets genuinely different options. When it does not, we do not pretend otherwise.

6. Discussion, Limitations, and Future Work

6.1 What the Hybrid Architecture Actually Buys

Across 317 tests, we have not seen a single overlap with a busy block, a self-overlap, an out-of-hours event, or a deadline violation. That is not because we tested every scenario. It is because the validator runs on every candidate every time, and the LLM is structurally blocked from overriding it. The LLM contribution stays exactly where it adds value: decomposing a vague goal into a sensible list, critiquing that list, reviewing three candidates from four perspectives, and writing rationales. We think this is the most reusable design lesson from the project: identify the boundary between what LLMs are good at and what pure functions are good at, and make that boundary explicit in the architecture.

6.2 Limitations

- Add-only writes. The agent cannot reschedule existing events. This is a deliberate safety boundary.
- Duration under-estimation on creative or open-ended goals persists despite the structural tag prompting. The user study confirms it.
- Single-week planning window. Multi-month plans would need day-aware spacing heuristics that we have not built yet.
- Real-LLM cost. Nine calls per nightly integration run still spends real tokens.

6.3 Where This Goes Next

Four directions from the original roadmap still feel productive.

Reject-and-retry: if the user dislikes all three candidates, we want to feed that rejection back ("the user wanted more morning blocks") and re-run, fully exploiting LangGraph's checkpointed state.

Rollback: a one-click undo using the write_results trace to delete what the agent just inserted, which is the only safe update or delete path because it cannot touch any user-authored events.

In-app customisation: let users drag tasks between slots after the proposal lands

Document-grounded planning: allow a user to upload a syllabus, course outline, or job description and ground the decomposition in that content rather than the LLM's prior alone.

The direction we would add, replacing the least important roadmap item: narrow the audience to students and fine-tune for them. Calendar planning across all possible goals is too broad to label cheaply. But if we focus on students (interview prep, assignment deadlines, exam prep), the input distribution becomes tractable.

A domain-specific fine-tuned model would likely outperform a general pretrained model on student tasks at significantly lower per-call cost. Going nicher is the standard playbook for boosting performance in a well-defined market, and we think it would pay off here.

7. Conclusion

We started with two research questions. The first asked how to divide responsibility between an LLM and deterministic code. Our answer: LLMs decompose goals, criticise plans, peer-review candidates,

and write rationales; pure Python schedules and validates. This ensures every category of LLM scheduling failure simply cannot reach the user, because the validator runs on every candidate before anything is surfaced. The second asked what contract the LLM should emit. Our answer is the three-tag structural contract: [group], [shuffle] and [complexity]. Small enough for a single prompt, expressive enough to drive three substantively different deterministic schedulers without re-querying the model. We think that contract is the architectural keystone.

The system delivers. 317 tests pass. 17 of 20 user-study participants rated the output four stars or above, with roughly 30 minutes of reported time savings per session. The hybrid pattern generalises: any task where natural language reasoning has to compose with strict, auditable constraints can benefit from drawing the same line.

8. Appendix

The questionnaire result is available in the link: https://forms.gle/tVUSq9SJ4cfEUG3f9